

* 코드리뷰 - 남윤우

섹션 단위 스크롤

```
function scrollToSectionSmooth(index) {  
  1 if (index < 0 || index >= allScrollSections.length) return;  
  2 isScrolling = true;  
  currentSection = index;  
  
  allScrollSections[index].scrollIntoView({  
    3 behavior: "smooth",  
    block: "start",  
  });  
  
  4 updateScrollToTopButton(index);  
  
  5 setTimeout(() => {  
    isScrolling = false;  
  }, 1000);  
}
```

1. 인덱스 유효성 검사
 - 섹션 범위를 벗어나는 요청은 무시
 - 첫 섹션보다 위로 가거나 마지막 섹션보다 아래로 가는 것 방지
2. 스크롤 중복 방지 플래그
 - isScrolling: 스크롤 애니메이션이 진행 중임을 표시
 - currentSection: 현재 위치한 섹션 인덱스 업데이트
3. 부드러운 스크롤 실행
 - scrollIntoView(): 브라우저 네이티브 API 사용
 - behavior: "smooth": CSS scroll-behavior와 동일한 부드러운 전환
 - block: "start": 해당 섹션이 뷰포트의 맨 위에 오도록 스크롤
4. 맨 위로 버튼 업데이트
 - 첫 번째 섹션(index === 0)이면 버튼 숨김
 - 그 외 섹션에서는 버튼 표시
5. 스크롤 잠금 해제
 - 1초(스크롤 애니메이션 시간) 후 플래그 해제
 - 이 시간 동안 다른 스크롤 입력 무시

* 코드리뷰 - 남윤우

휠로 섹션 이동

```
1 const handleWheel = (e) => {  
  // 모달이 열려있으면 스크롤 막기  
  if (document.body.classList.contains('modal-open')) {  
    e.preventDefault();  
    return;  
  }  
2 e.preventDefault();  
3 if (isScrolling) return;  
4 if (e.deltaY > 0 && currentSection < allScrollSections.length - 1) {  
  scrollToSectionSmooth(currentSection + 1);  
} else if (e.deltaY < 0 && currentSection > 0) {  
  scrollToSectionSmooth(currentSection - 1);  
}  
};
```

1. 모달 감지

- 모달이 열려있으면 풀페이지 스크롤 비활성화
- 사용자가 모달 내용을 자유롭게 스크롤 가능

2. 기본 스크롤 동작 차단

- 브라우저의 일반적인 스크롤 막기
- 우리가 정의한 섹션 단위 스크롤만 동작

3. 스크롤 중복 방지

- 스크롤 애니메이션이 진행 중이면 새로운 입력 무시
- 빠르게 여러 번 휠을 돌려도 한 섹션씩만 이동

4. 휠 방향 감지

- $e.deltaY > 0$: 휠을 아래로 (다음 섹션으로)
- $e.deltaY < 0$: 휠을 위로 (이전 섹션으로)
- 경계 체크: 첫/마지막 섹션에서는 더 이상 이동 안 함

* 코드리뷰 - 남윤우

스크롤 시 헤더 색상 및 섹션 위치 감지

```
const handleScroll = () => {  
  1 if (!mainSection || !searchSection || !header) return;  
  
  2 const searchSectionEnd =  
    searchSection.offsetTop + searchSection.offsetHeight;  
  
  3 if (window.scrollY >= searchSectionEnd - 100) {  
    header.classList.add("scrolled");  
  } else {  
    header.classList.remove("scrolled");  
  }  
}
```

기능 1 : 헤더 배경색 변경

1. DOM 요소 확인

- 필요한 요소들이 로드되지 않았으면 실행 안 함

2. 검색 섹션 끝 지점 계산

- offsetTop: 섹션의 시작 위치 (페이지 상단부터의 거리)
- offsetHeight: 섹션의 높이
- 합: 검색 섹션이 끝나는 Y 좌표

3. 헤더 스타일 토글

- window.scrollY: 현재 스크롤 위치
- 검색 섹션 끝에서 100px 전부터 헤더에 scrolled 클래스 추가
- CSS에서 .scrolled 클래스로 배경색, 텍스트 색상 등 변경

CSS 연동 (Header.css):

```
header.scrolled {  
  background: white;  
  color: #333;  
}
```

* 코드리뷰 - 남윤우

스크롤 시 헤더 색상 및 섹션 위치 감지

```
1 if (isScrolling) return;

2 let current = 0;
  const scrollPosition = window.pageYOffset; // 현재 스크롤 위치
  const windowHeight = window.innerHeight; // 브라우저 창 높이
  const documentHeight = document.documentElement.scrollHeight;
                                     // 전체 문서 높이

3 allScrollSections.forEach((section, index) => {
  if (!section) return;
  const sectionTop = section.offsetTop; // 섹션 시작 위치
  const sectionHeight = section.clientHeight; // 섹션 높이
  const checkPosition = scrollPosition + windowHeight / 2;
                                     // 화면 중앙 유지

  if (
    checkPosition >= sectionTop &&
    checkPosition < sectionTop + sectionHeight
  ) {
    current = index;
  }
});

4 if (scrollPosition + windowHeight >= documentHeight - 50) {
  current = allScrollSections.length - 1;
}

5 updateScrollToTopButton(current);
  currentSection = current;
};
```

기능 2: 현재 섹션 위치 추적

1. 스크롤 중 위치 추적 스킵

- 섹션 단위 스크롤 애니메이션 중에는 위치 추적 안 함
- 애니메이션이 끝난 후에만 정확한 위치 계산

2. 필요한 값 계산

3. 각 섹션 순회하며 현재 위치 판단

- 현재 화면 중앙이 어느 섹션에 있는지 판단

4. 페이지 하단 감지

- 페이지 맨 아래에 도달하면 마지막 섹션으로 인식
- -50: 오차 범위 (정확히 끝까지 안 가도 인식)

5. 상태 업데이트

updateScrollToTopButton(current); // 버튼 표시/숨김
currentSection = current; // 현재 섹션 인덱스 저장

